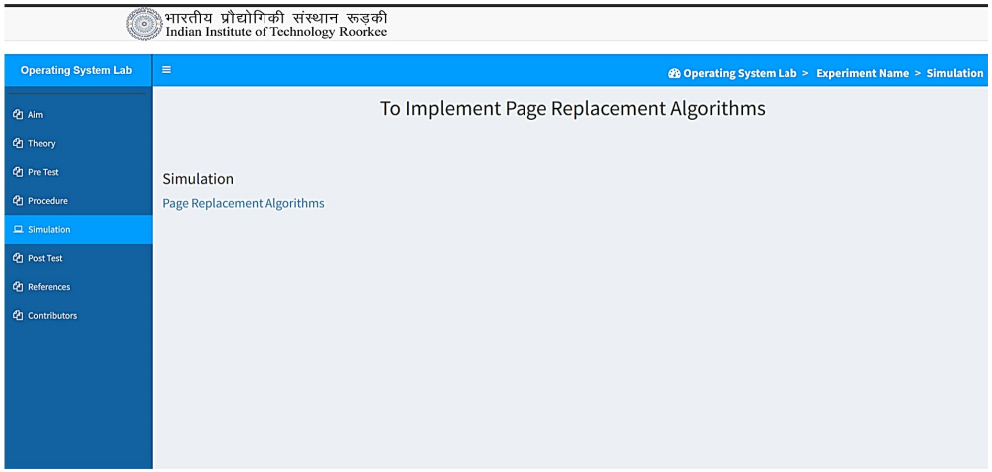
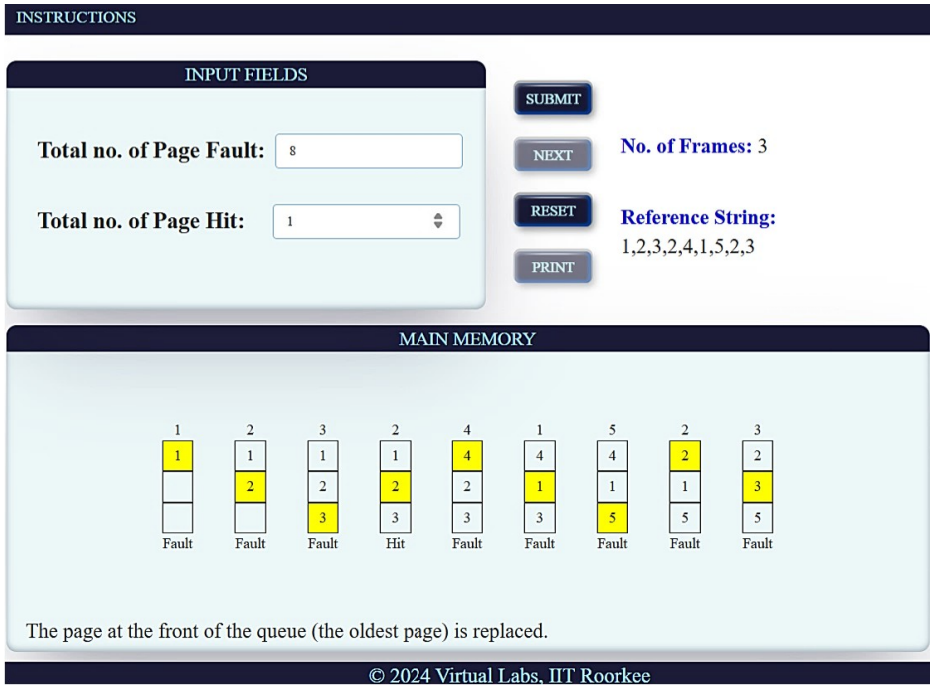
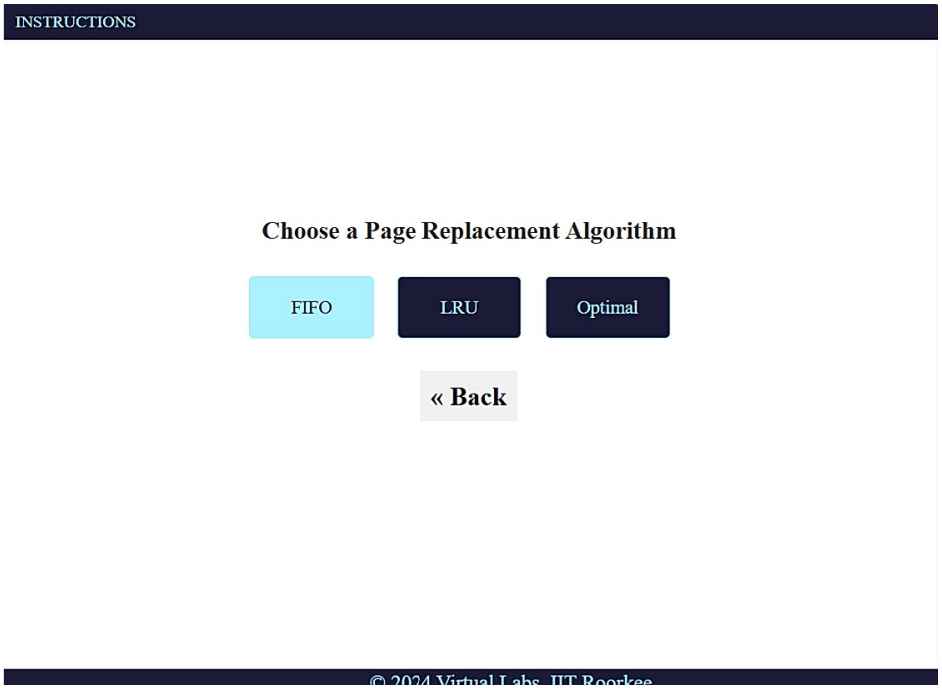

भारतीय प्रौद्योगिकी संस्थान रूड़की
Indian Institute of Technology Roorkee
Operating System Lab
Operating System Lab > Experiment Name > Simulation
Aim
Theory
Pre Test
Procedure
Simulation
Post Test
References
Contributors
To Implement Page Replacement Algorithms
Simulation
Page Replacement Algorithms


INSTRUCTIONS
INPUT FIELDS
No. of Frames: 3
Reference String: 1, 2, 3, 2, 4, 1, 5, 2,3
SUBMIT
NEXT
RESET
PRINT
MAIN MEMORY
© 2024 Virtual Labs, IIT Roorkee


INSTRUCTIONS
INPUT FIELDS
Total no. of Page Fault: 8
Total no. of Page Hit: 1
SUBMIT
NEXT
RESET
PRINT
No. of Frames: 3
Reference String:
1,2,3,2,4,1,5,2,3
MAIN MEMORY
1 2 3 2 4 1 5 2 3
Fault Fault Fault Hit Fault Fault Fault Fault Fault
The page at the front of the queue (the oldest page) is replaced.
© 2024 Virtual Labs, IIT Roorkee


INSTRUCTIONS
Choose a Page Replacement Algorithm
FIFO
LRU
Optimal
« Back
© 2024 Virtual Labs, IIT Roorkee

ass3 Q1

```cpp
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

// Function to print results and calculate metrics
void displayResults(const string& strategyName,
                    const vector<int>& processSize,
                    const vector<int>& allocation,
                    const vector<int>& remainingBlockSize,
                    int initialTotalBlockMem) {

    int m = processSize.size();
    int allocatedCount = 0;
    int allocatedMemorySum = 0;

    cout << "\n================ " << strategyName << " Allocation Table ================\n";
    cout << left << setw(12) << "Process No."
         << setw(16) << "Process Size"
         << setw(14) << "Block Allocated" << endl;
    cout << "-----------------------------------------------------------\n";

    for (int i = 0; i < m; i++) {
        cout << left << setw(12) << (i + 1)
             << setw(16) << processSize[i];
        if (allocation[i] != -1) {
            cout << "Block " << (allocation[i] + 1) << endl;
            allocatedCount++;
            allocatedMemorySum += processSize[i];
        } else {
            cout << "Not Allocated" << endl;
        }
    }

    int remainingFreeMemory = 0;
    for (int size : remainingBlockSize) {
        remainingFreeMemory += size;
    }

    double memoryUtilization = ((double)allocatedMemorySum / initialTotalBlockMem) * 100.0;

    cout << "-----------------------------------------------------------\n";
    cout << "Successfully Allocated Processes : " << allocatedCount << " / " << m << endl;
    cout << "Remaining Free Memory            : " << remainingFreeMemory << " KB" << endl;
    cout << fixed << setprecision(2);
```

```
    cout << "Memory Utilization               : " << memoryUtilization << " %" <<
endl;
}

// 1. First Fit Algorithm
void firstFit(vector<int> blockSize, vector<int> processSize, int totalBlockMem) {
    int m = processSize.size();
    int n = blockSize.size();
    vector<int> allocation(m, -1);

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (blockSize[j] >= processSize[i]) {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }

    displayResults("First Fit", processSize, allocation, blockSize, totalBlockMem);
}

// 2. Best Fit Algorithm
void bestFit(vector<int> blockSize, vector<int> processSize, int totalBlockMem) {
    int m = processSize.size();
    int n = blockSize.size();
    vector<int> allocation(m, -1);

    for (int i = 0; i < m; i++) {
        int bestIdx = -1;
        for (int j = 0; j < n; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }
        }

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }

    displayResults("Best Fit", processSize, allocation, blockSize, totalBlockMem);
}

// 3. Worst Fit Algorithm
void worstFit(vector<int> blockSize, vector<int> processSize, int totalBlockMem) {
    int m = processSize.size();
    int n = blockSize.size();
    vector<int> allocation(m, -1);

```

```
    for (int i = 0; i < m; i++) {
        int worstIdx = -1;
        for (int j = 0; j < n; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
                    worstIdx = j;
                }
            }
        }

        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blockSize[worstIdx] -= processSize[i];
        }
    }

    displayResults("Worst Fit", processSize, allocation, blockSize, totalBlockMem);
}

int main() {
    // Memory blocks available in the system
    vector<int> blockSize = {100, 500, 200, 300, 600};

    // Process memory requests
    vector<int> processSize = {212, 417, 112, 426};

    int totalBlockMem = 0;
    for (int size : blockSize) {
        totalBlockMem += size;
    }

    cout << "Total Physical Memory Available: " << totalBlockMem << " KB\n";
    cout << "Blocks Available: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB\n";
    cout << "Process Requests: P1(212 KB), P2(417 KB), P3(112 KB), P4(426 KB)\n";

    // Run each strategy on a copy of the block sizes
    firstFit(blockSize, processSize, totalBlockMem);
    bestFit(blockSize, processSize, totalBlockMem);
    worstFit(blockSize, processSize, totalBlockMem);

    return 0;
}
```

```
Total Physical Memory Available: 1700 KB
Blocks Available: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB
Process Requests: P1(212 KB), P2(417 KB), P3(112 KB), P4(426 KB)

================ First Fit Allocation Table ================
Process No. Process Size    Block Allocated
-----------------------------------------------------------
1           212             Block 2
2           417             Block 5
3           112             Block 2
4           426             Not Allocated
-----------------------------------------------------------
Successfully Allocated Processes : 3 / 4
Remaining Free Memory            : 959 KB
Memory Utilization               : 43.59 %

================ Best Fit Allocation Table ================
Process No. Process Size    Block Allocated
-----------------------------------------------------------
1           212             Block 4
2           417             Block 2
3           112             Block 3
4           426             Block 5
-----------------------------------------------------------
Successfully Allocated Processes : 4 / 4
Remaining Free Memory            : 533 KB
Memory Utilization               : 68.65 %

================ Worst Fit Allocation Table ================
Process No. Process Size    Block Allocated
-----------------------------------------------------------
1           212             Block 5
2           417             Block 2
3           112             Block 5
4           426             Not Allocated
-----------------------------------------------------------
Successfully Allocated Processes : 3 / 4
Remaining Free Memory            : 959 KB
Memory Utilization               : 43.59 %
```

ass3 Q2

```cpp
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

// Helper function to print the current state of frames
void printFrames(const vector<int>& frames) {
    for (int page : frames) {
        if (page != -1) {
            cout << page << " ";
        } else {
            cout << "- ";
        }
    }
}

// 1. First-In, First-Out (FIFO)
void fifo(const vector<int>& pages, int frameCount) {
    vector<int> frames(frameCount, -1);
    int pageFaults = 0;
    int pageHits = 0;
    int insertIndex = 0;

    cout << "\n================ FIFO Page Replacement ================\n";
    cout << left << setw(8) << "Ref"
         << setw(16) << "Frames"
         << setw(10) << "Status" << endl;
    cout << "-------------------------------------------------------\n";

    for (int page : pages) {
        bool hit = false;
        for (int f : frames) {
            if (f == page) {
                hit = true;
                break;
            }
        }

        cout << left << setw(8) << page;

        if (hit) {
            pageHits++;
            cout << setw(16);
            printFrames(frames);
            cout << setw(10) << "Hit" << endl;
        } else {
            pageFaults++;
            frames[insertIndex] = page;
            insertIndex = (insertIndex + 1) % frameCount;
```

```
            cout << setw(16);
            printFrames(frames);
            cout << setw(10) << "Fault" << endl;
        }
    }

    cout << "-------------------------------------------------------\n";
    cout << "Total Page Faults : " << pageFaults << endl;
    cout << "Total Page Hits   : " << pageHits << endl;
}

// 2. Least Recently Used (LRU)
void lru(const vector<int>& pages, int frameCount) {
    vector<int> frames(frameCount, -1);
    vector<int> lastUsed(frameCount, -1);
    int pageFaults = 0;
    int pageHits = 0;

    cout << "\n================ LRU Page Replacement ================\n";
    cout << left << setw(8) << "Ref"
         << setw(16) << "Frames"
         << setw(10) << "Status" << endl;
    cout << "-------------------------------------------------------\n";

    for (int t = 0; t < (int)pages.size(); t++) {
        int page = pages[t];
        int frameIdx = -1;

        for (int i = 0; i < frameCount; i++) {
            if (frames[i] == page) {
                frameIdx = i;
                break;
            }
        }

        cout << left << setw(8) << page;

        if (frameIdx != -1) {
            pageHits++;
            lastUsed[frameIdx] = t;
            cout << setw(16);
            printFrames(frames);
            cout << setw(10) << "Hit" << endl;
        } else {
            pageFaults++;
            // Check if there is an empty frame first
            int replaceIdx = -1;
            for (int i = 0; i < frameCount; i++) {
                if (frames[i] == -1) {
                    replaceIdx = i;
                    break;
                }
            }

```

```
            // If no frame is empty, pick the least recently used
            if (replaceIdx == -1) {
                int leastTime = lastUsed[0];
                replaceIdx = 0;
                for (int i = 1; i < frameCount; i++) {
                    if (lastUsed[i] < leastTime) {
                        leastTime = lastUsed[i];
                        replaceIdx = i;
                    }
                }
            }

            frames[replaceIdx] = page;
            lastUsed[replaceIdx] = t;
            cout << setw(16);
            printFrames(frames);
            cout << setw(10) << "Fault" << endl;
        }
    }

    cout << "-------------------------------------------------------\n";
    cout << "Total Page Faults : " << pageFaults << endl;
    cout << "Total Page Hits   : " << pageHits << endl;
}

// 3. Optimal Page Replacement
void optimal(const vector<int>& pages, int frameCount) {
    vector<int> frames(frameCount, -1);
    int pageFaults = 0;
    int pageHits = 0;

    cout << "\n================ Optimal Page Replacement ================\n";
    cout << left << setw(8) << "Ref"
         << setw(16) << "Frames"
         << setw(10) << "Status" << endl;
    cout << "-------------------------------------------------------\n";

    for (int t = 0; t < (int)pages.size(); t++) {
        int page = pages[t];
        bool hit = false;

        for (int f : frames) {
            if (f == page) {
                hit = true;
                break;
            }
        }

        cout << left << setw(8) << page;

        if (hit) {
            pageHits++;
            cout << setw(16);
            printFrames(frames);
```

```
            cout << setw(10) << "Hit" << endl;
        } else {
            pageFaults++;

            int replaceIdx = -1;
            for (int i = 0; i < frameCount; i++) {
                if (frames[i] == -1) {
                    replaceIdx = i;
                    break;
                }
            }

            // If full, find page that will not be used for longest time in future
            if (replaceIdx == -1) {
                int farthest = t;
                for (int i = 0; i < frameCount; i++) {
                    int nextUse = -1;
                    for (int j = t + 1; j < (int)pages.size(); j++) {
                        if (frames[i] == pages[j]) {
                            nextUse = j;
                            break;
                        }
                    }

                    // If never referenced again in future, replace immediately
                    if (nextUse == -1) {
                        replaceIdx = i;
                        break;
                    }

                    if (nextUse > farthest) {
                        farthest = nextUse;
                        replaceIdx = i;
                    }
                }
            }

            frames[replaceIdx] = page;
            cout << setw(16);
            printFrames(frames);
            cout << setw(10) << "Fault" << endl;
        }
    }

    cout << "-------------------------------------------------------\n";
    cout << "Total Page Faults : " << pageFaults << endl;
    cout << "Total Page Hits   : " << pageHits << endl;
}

int main() {
    // Standard reference string from Question 5 sequence: 1, 2, 3, 2, 4, 1, 5, 2
    vector<int> pages = {1, 2, 3, 2, 4, 1, 5, 2};
    int frameCount = 3;

```

```
    cout << "Page Reference String : 1, 2, 3, 2, 4, 1, 5, 2\n";
    cout << "Total Page Frames     : 3\n";

    fifo(pages, frameCount);
    lru(pages, frameCount);
    optimal(pages, frameCount);

    return 0;
}
```

```
Page Reference String : 1, 2, 3, 2, 4, 1, 5, 2
Total Page Frames     : 3

================ FIFO Page Replacement ================
Ref     Frames          Status
-------------------------------------------------------
1       1 - -           Fault
2       1 2 -           Fault
3       1 2 3           Fault
2       1 2 3           Hit
4       4 2 3           Fault
1       4 1 3           Fault
5       4 1 5           Fault
2       2 1 5           Fault
-------------------------------------------------------
Total Page Faults : 7
Total Page Hits   : 1

================ LRU Page Replacement ================
Ref     Frames          Status
-------------------------------------------------------
1       1 - -           Fault
2       1 2 -           Fault
3       1 2 3           Fault
2       1 2 3           Hit
4       1 2 4           Fault
1       1 2 4           Hit
5       1 2 5           Fault
2       1 2 5           Hit
-------------------------------------------------------
Total Page Faults : 5
Total Page Hits   : 3

================ Optimal Page Replacement ================
Ref     Frames          Status
-------------------------------------------------------
1       1 - -           Fault
2       1 2 -           Fault
3       1 2 3           Fault
2       1 2 3           Hit
4       1 2 4           Fault
1       1 2 4           Hit
5       5 2 4           Fault
2       5 2 4           Hit
-------------------------------------------------------
Total Page Faults : 5
Total Page Hits   : 3
```